

Support Ticket System Design

A project-first Java learning blueprint
Console application to Spring Boot REST APIs

Prepared for Yadu Krishnan | 05 October 2026 | Design version 1.0

Project purpose

Customers report issues. Administrators assign tickets to support agents. Agents discuss the issue, provide a resolution and record progress. Customers confirm closure or reopen an unresolved issue.

Example workflow

A customer reports "Unable to log in". An admin assigns the ticket. The agent starts work, adds comments and provides a solution. The customer confirms that the issue is fixed and closes the ticket.

Implementation approach

Start with a single plain Java console application. Keep business rules in services and storage behind a repository interface. Introduce collections, JDBC and REST APIs in stages. A distributed microservice architecture is outside the initial scope.

Stage	Deliverable
Core Java	Console application with in-memory tickets
SQL + JDBC	Persistent tickets, comments and history in MySQL
Spring Boot	REST APIs, validation, authentication and later JPA
Extension	React UI, Docker/AWS deployment and optional GenAI

First milestone

Create a Ticket object, initialize its status to OPEN and display its details. Add one concept at a time and explain why the project needs it.

1. Requirements and access rules

Role	Allowed actions
Customer	Create tickets; view own tickets; comment; close or reopen own resolved tickets.
Support Agent	View assigned tickets; comment; start work; resolve assigned tickets.
Admin	View all tickets; assign/reassign agents; change priority; manage users.

Core features

Create tickets; find by ID; list and filter by status, priority or assigned agent; assign an agent; update status; add comments; view status history. REST lists use pagination.

Data and defaults

Priorities: LOW, MEDIUM, HIGH, URGENT. Statuses: OPEN, IN_PROGRESS, RESOLVED, CLOSED. The initial status is OPEN; an agent may be unassigned. No ticket deletion is included in version 1 so history remains available.

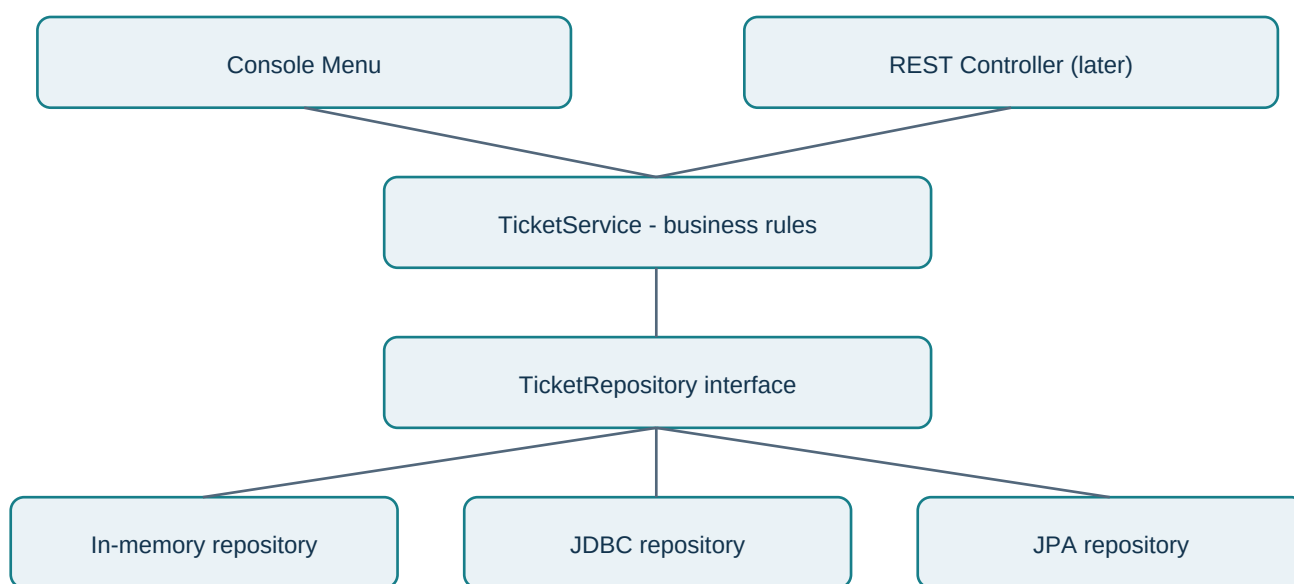
Operation	Business rule
Create	Title and description are required. Creator becomes ticket owner.
Assign	Admin action. Target user must be a support agent. Reassignment preserves ticket status.
Start work	Only the assigned agent; an assignment must exist.
Resolve	Only the assigned agent; a nonblank resolution summary is required.
Close / reopen	Only the owner, and only from RESOLVED.
Comment	Owner, assigned agent or admin; reject comments on CLOSED tickets.
Status update	Allow only documented transitions and create a history record.

Console development uses sample users and passes the acting user to services. In the REST phase, derive identity from authentication and enforce permissions in the service layer. Do not trust a client-supplied user ID as proof of identity.

2. Lifecycle and application architecture

From	To	Actor / condition
New	OPEN	Customer creates ticket
OPEN	IN_PROGRESS	Assigned agent starts work
IN_PROGRESS	RESOLVED	Assigned agent provides resolution
RESOLVED	CLOSED	Owner confirms resolution
RESOLVED	IN_PROGRESS	Owner reopens; assigned agent must exist

Assignment and status are separate: assigning an agent does not automatically start work. CLOSED is terminal in this version. Reject all transitions absent from the table.



JDBC / JPA persist to MySQL

Layer	Responsibility
Console / Controller	Accept input and present output; REST controllers handle HTTP.
Service	Validate actions, enforce access and status rules, coordinate transactions.
Repository	Save, retrieve and query data; isolate storage details.
Model / Database	Represent domain data / persist it across restarts.

Memory, JDBC and JPA are successive storage implementations. They are not three stores that must run together. Preserve business rules while replacing persistence.

3. Domain model and database

Class / interface	Purpose
User	Identity, name, email and role
Ticket	Issue details, priority, status, owner, assigned agent and timestamps
TicketComment	Ticket message, author and timestamp
TicketStatusHistory	Old/new status, acting user and timestamp
TicketService	Business operations and validation
TicketRepository	Contract for persistence operations
InMemoryTicketRepository	Collections-based implementation
ConsoleMenu / Main	User interaction / application entry point

Use enums `UserRole`, `TicketStatus` and `TicketPriority`. Model user permissions with a role rather than forcing a Customer/Agent subclass hierarchy. For inheritance practice, a Notification abstraction can have `ConsoleNotification` and later `EmailNotification` implementations.

Table	Main columns
users	id (PK), name, email (UNIQUE), role, created_at; password_hash in authentication phase
tickets	id (PK), title, description, priority, status, customer_id (FK), assigned_agent_id (nullable FK), resolution_summary, created_at, updated_at
ticket_comments	id (PK), ticket_id (FK), author_id (FK), message, created_at
ticket_status_histories	id (PK), ticket_id (FK), from_status (nullable for creation), to_status, changed_by (FK), changed_at

Relationships: One customer owns many tickets; one agent handles many tickets; each ticket has many comments and status-history records. Foreign keys reference users and tickets. Do not cascade-delete audit history through routine user actions.

Transaction: Update ticket status and insert the history record atomically. If either fails, roll back both. Use prepared statements in JDBC and try-with-resources to close connections, statements and result sets.

4. REST API and operation flow

Method	Endpoint	Purpose
POST	/api/v1/tickets	Create
GET	/api/v1/tickets	List / filter / paginate
GET	/api/v1/tickets/{id}	View one
PATCH	/api/v1/tickets/{id}/assignment	Assign / reassign
PATCH	/api/v1/tickets/{id}/status	Transition status
POST	/api/v1/tickets/{id}/comments	Add comment
GET	/api/v1/tickets/{id}/comments	List comments
GET	/api/v1/tickets/{id}/history	View history

Create ticket flow

1. Read title, description and priority.
2. Identify the acting customer.
3. Validate required values and priority.
4. Set status OPEN and record the owner/time.
5. Save the ticket and its initial history together.
6. Return the created ticket; REST phase returns HTTP 201.

Status update flow

Load ticket; check acting user; validate transition; require a resolution summary when resolving; update ticket and append history in a transaction; return the updated ticket.

HTTP status	Meaning
200 / 201	Successful read/update / creation
400	Malformed input or validation failure
401 / 403	Unauthenticated / insufficient permissions
404	Ticket does not exist
409	Requested operation conflicts with current ticket state

Implementation quality

Use request/response DTOs in REST, validate inputs, hash passwords through the authentication framework and omit password hashes from responses. Add meaningful checks for invalid transitions, unauthorized actions and transactional persistence. Later guard concurrent updates with optimistic locking to avoid stale changes.

5. Learning roadmap and extensions

Phase	Concepts	Project exercise
1	Classes, objects, constructors, encapsulation, enums	Create/display a ticket; control field access.
2	Interfaces, abstraction, polymorphism, inheritance	Repository contract and notification implementations.
3	Exception handling	TicketNotFoundException and InvalidStatusTransitionException.
4	Collections + basic generics	Map<Long, Ticket> for lookup; lists for comments/history.
5	Generics in depth	Reusable PageResult<T> for paginated results.
6	Java 8	Lambdas, functional interfaces, Streams, Optional and LocalDateTime.
7	SQL + JDBC	Tables, joins, prepared statements, transactions and resource cleanup.
8	Spring Boot + REST	Dependency injection, controllers, DTOs, validation, security and later JPA.

How each lesson works

Explain the concept, why this project needs it, its code execution, an interview explanation and a small independent task. Start filtering with loops; introduce Streams after the loop-based logic is understood. Explain basic generics when collections first require them.

Milestones

- M1: Create/display a ticket.
- M2: Store multiple tickets, look up by ID and validate input.
- M3: Assign agents, enforce transitions, add comments/history.
- M4: Persist and reload all data with JDBC.
- M5: Expose secured REST APIs and migrate persistence to JPA.

Optional GenAI extension

Summarize conversations, suggest categories and draft agent replies. Agents review drafts before sending. The AI service is optional: failures must not block ticket operations. Document-based answers can later include source references.

Later delivery scope

Add a React frontend, containerize the application and deploy to AWS after the backend works. Keep AI, frontend and deployment separate from the first Core Java milestone.